시스템 프로그래밍

Proxy 1- 1

담당교수 : 최상호 교수님

학번 : 2021202089

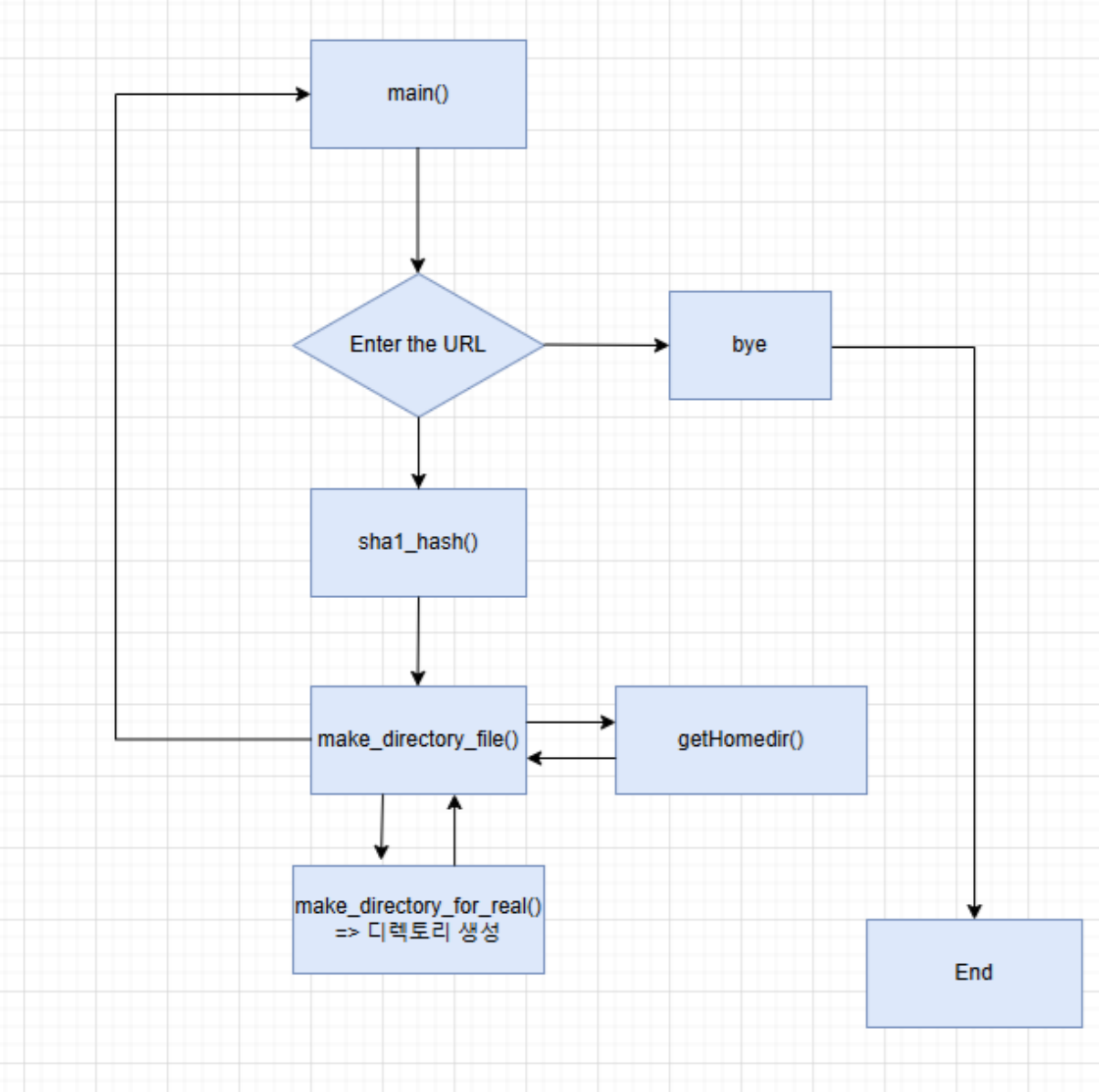
이름 : 오나균

1.Introduction

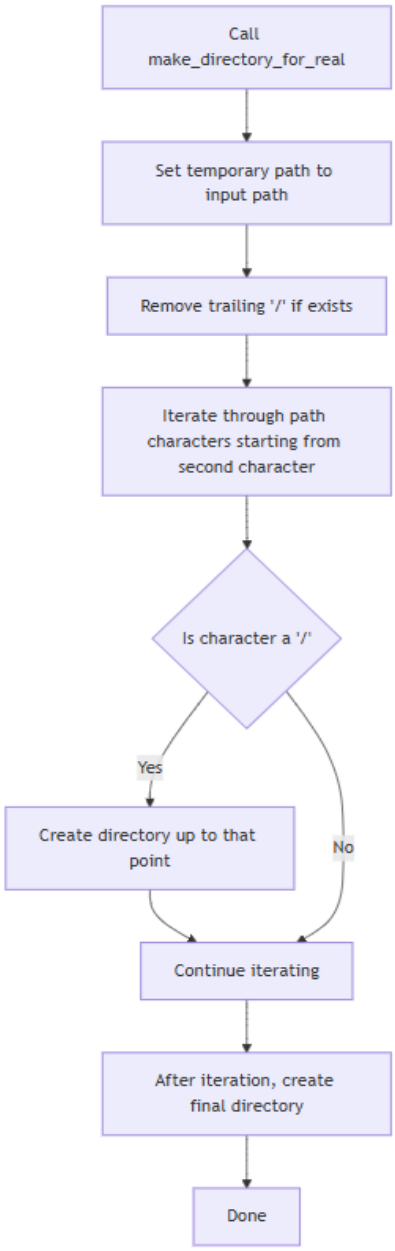
우선 이번 과제는 2가지가 나왔는데, 첫번째로는 mkdir()을 이용하여, 디렉토리 생성시 발생하는 간단한 권한 문제를 해결 하는 것과 두번째로는 URL을 입력했을 때, 과제에서 기본적으로 주어진 sha1_hash함수, getHomedir함수들을 이용하여, URL을 Hashed URL로 변환한 뒤, Hashed 된 URL에 해당하는 directory 와 file을 생성하여 저장하는 것이다. 두번째 과제에 대한 전체적인 진행과정을 정리해보면, 1) C코드 작성 , 2.) Makefile작성, 3) make를 통한 컴파일, 4)생성된 파일 실행 , 5) tree 명령어를 통한 결과 확인으로 나누어진다.

Flow Chart

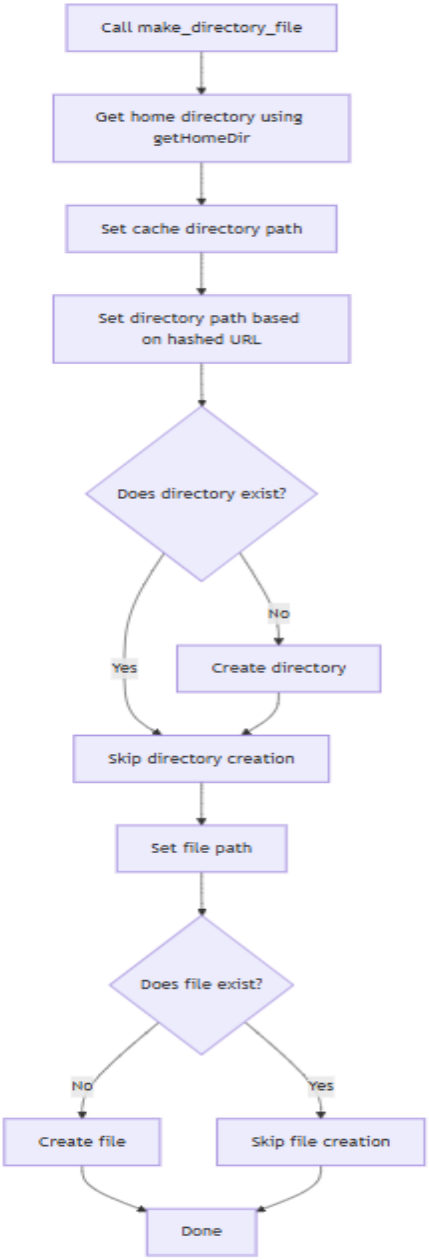
⇨ **Main()**



⇨ **make_directory_for_real()**



⇨ **make_directory_file**



2.Pseudo code

Make_directory_for_real()

1. 경로(path)를 받아서 temp 배열에 저장한다.
2. 경로의 길이를 계산한다.
3. 경로의 마지막에 '/'가 있으면 제거한다.
4. temp 배열을 처음부터 끝까지 반복하면서:

   =>/ 를 만나면, 그 지점에서 temp 문자열을 끊어 중간 디렉토리를 만든다.

   => 중간 디렉토리를 만든 후, 다시 /를 복구하여 원래 경로를 유지한다.
5. 모든 중간 디렉토리를 만든 후, 최종 디렉토리를 만든다.

Make_directory_file()

1. home 디렉토리를 가져온다.
2. home 디렉토리에 /cache를 추가하여 기본 캐시 디렉토리 경로를 만든다.
3. 해시된 URL의 첫 번째, 두 번째, 세 번째 문자를 기반으로 하위 디렉토리 경로를 만든다.
4. 해당 디렉토리가 존재하지 않으면:

   => Make_directory_for_real() 함수를 사용하여 디렉토리를 만든다.
5. 해시된 URL의 나머지 부분을 사용하여 파일 경로를 만든다.
6. 해당 파일이 존재하지 않으면:

   a. fopen()을 사용하여 새 파일을 생성하고, 생성된 파일을 닫는다.

main()

1. URL입력 (bye 입력시 종료)
2. sha1_hash(url,hashed_url)함수를 통해 hashed_url을 얻어냄
3. Make_directory_file(hashed_url) 함수를 통해 hashed_url로 구성된 디렉토리 및 파일 생성
4. while(1)

3. 결과화면

1) mkdir()

⇨ 발생한 이유 : mkdir() 함수에서 directory를 생성할 때, 지정된 권한을 그대로 적용하게끔 만드려면 **umask(umask란 새로 생성되는 파일이나 디렉토리 권한 제한하는 명령어)**를 고려해야 합니다.

```
kw2021202089@ubuntu:~$ umask
0002
```

⇨ umask => 0002

```
kw2021202089@ubuntu: ~
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>

void main(int argc,char* argv[])
{
        if(argc<2){
                printf("error\n");
                return;
        }
        mkdir(argv[1],S_IRWXU|S_IRWXG|S_IRWXO);

}
```

⇨ 권한설정 0777

```
kw2021202089@ubuntu:~$ vim ex.c
kw2021202089@ubuntu:~$ gcc -o test ex.c
kw2021202089@ubuntu:~$ ./test test_directory
kw2021202089@ubuntu:~$ ls -ld test_directory
drwxrwxr-x 2 kw2021202089 kw2021202089 4096 Mar 29 02:19 test_directory
kw2021202089@ubuntu:~$
```

⇨ umask로 인해0777 – 0002 = 0775 로 권한이 설정된 것을 알 수 있습니다.

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h> // umask (0)

void main(int argc,char* argv[])
{
        if(argc<2){
                printf("error\n");
                return;
        }
        umask(0);

        mkdir(argv[1],S_IRWXU|S_IRWXG|S_IRWXO);

}
```

⇨ 수정된 코드 , umask(0)으로 설정하여 올바르게 권한을 부여하게 함.

```
kw2021202089@ubuntu:~$ gcc -o fix ex.c
kw2021202089@ubuntu:~$ ./fix fix_directory
kw2021202089@ubuntu:~$ ls -ld fix_directory
drwxrwxrwx 2 kw2021202089 kw2021202089 4096 Mar 29 02:25 fix_directory
kw2021202089@ubuntu:~$
```

권한이 0777 -0000 = 0777로 설정되었습니다.

2) Proxy # 1- 1 Requirements

```
kw2021202089@ubuntu:~$ ls
Desktop    Downloads  Music     proxy_cache.c  splab_commands  Videos
Documents  Makefile   Pictures  Public         Templates       work
kw2021202089@ubuntu:~$ make
gcc -o proxy_cache proxy_cache.c -lcrypto
kw2021202089@ubuntu:~$ ls
Desktop    Downloads  Music     proxy_cache    Public          Templates  work
Documents  Makefile   Pictures  proxy_cache.c  splab_commands  Videos
kw2021202089@ubuntu:~$ ./proxy_cache
input url>www.kw.ac.kr
input url>www.google.com
input url>www.naver.com
input url>klas.kw.ac.kr
input url>bye
kw2021202089@ubuntu:~$ tree ~/cache/
/home/kw2021202089/cache/
├── 3ef
│   └── 9fd210fb8e00c8114ff978d282258ed8a48ea
├── d8b
│   └── 99f68b208b5453b391cb0c6c3d6a9824f3c3a
├── e00
│   └── 0f293fe62e97369e4b716bb3e78fababf8f90
└── fed
    └── 818da7395e30442b1dcf45c9b6669d1c0ff6b

4 directories, 4 files
kw2021202089@ubuntu:~$
```

- 만들어둔 Makefile과 proxy_cache.c 파일이 make 명령어를 통해 컴파일 된다.
- ls 명령어를 통해 proxy_cache 파일이 생성된 것을 알 수 있다.
- ./proxy_cache 명령어를 통해 c파일을 실행
- 예제와 마찬가지로 www.kw.ac.kr , www.google.com , www.naver.com , klas.kw.ac.kr, 이 입력된 뒤 bye명령어를 통해 종료
- tree 명령어를 통해 home/kw2021202089/cache/(hashed_url의 앞 세글자)/(hashed 된파일이름) 이 4 directories, 4 files 생성된 것을 알 수있다.

4. 고찰

이번 1차 과제를 통해서, 우선 첫번째 문제를 통해서는 directory()의 권한이 제대로 할당 되지 않을 때에는 기존에 설정된 umask에 의해 영향을 받아서 그런 문제가 발생한다는 것을 알게 되었다.

다음으로 두번째 문제, 이게 사실 고비였다. 리눅스에 익숙하지 않아서, 실습 자료에서 설명해주는 이 텍스트가 어떤 행동으로 이어져야 되는지에 대해서 잘 떠오르지 않는 그런 어려움을 많이 느꼈다.

처음에 잘 와닿지 않았던 것 중에 하나는 "c파일과 Makefile을 만든뒤 make를 통해서 컴파일 후 실행하라" 에 대한 것이다. 지금 와서는 이렇게 한 문장으로 정리할 수 있지만, 과제 초반에는 이 한문장을 내 뇌에서 온전히 받아들이는 것이 쉽지 않았다.

사실 첫번째 Basic 과제를 휘몰아쳐서 하는 느낌이 없지 않아 있었던 것 때문인지, 맨 처음에 개념 이해를 잘못 하고 넘어간 것 같았다. 그래서 스스로 학습하며 알게 된 점은 우선 리눅스에서 Makefile, makefile 이라는 파일명을 이미 이런 기능에서 고정해둔 점이다. 그 후 make를 통한 과정이 그냥 기본 개념? 이란 점을 알게 되서 납득을 할 수 있게 되었다. 아 그리고 make가 Makefile에 대한 자동화된 컴파일 명령어 기능을 수행해서 이를 가능케 한다는 것을 알게 되었다.

그래서 c 파일을 만들고, 해당 c파일에 대해서 , Makefile을 작성한 뒤에 , make 명령어 (컴파일)을 하면 , Makefile 명령어에서 정해놓은 실행 파일명을 (./실행파일명) 치면 코드가 실행 되는 구조이다.

또한 이 과제를 하면서 헷갈렸던 개념을 명확히 한 것 같은데, 바로 2자리 16진수의 1자리는 4비트를 표현해서, 총 8비트, 즉 1byte를 차지한다는 점이다. 그래서 이 문제에서 sha1_hash함수의 주어진 코드에서 20byte x 2 = 40 (1byte = 8bit = 4bit x 2) 크기가 40인 hashed_url 에 할당된다는 코드가 있는 것이다.

처음에 pdf를 볼 때는 리눅스를 거의 처음 접해 봐서 인지, 무슨 말인지 이해도 안가고, 이 vi 에디터를 쓰는데도 불편해가지고 쉽지 않았다. 그래도 실습 강의자료를 읽어보고, 인터넷에 검색해서 왜 안되는지를 알아가면서, 처음보다는 그래도 리눅스를 잘 쓸 수 있게 된 것 같았다.

5. Reference

https://m.blog.naver.com/doctor-kick/222059747063 (umask)

https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=silverstonec&logNo=60196870427